AgentPPprof: Operation-Stack Profiling for AI Agent Traces

Abstract

AI-agent executions are no longer well described by one prompt, one session, or one trace tree. A single run can contain prompts, tool calls, GUI actions, browser actions, process events, plans, subagents, safety labels, failure labels, and human subtask boundaries. Existing observability systems usually make one boundary primary, such as a session, span, or prompt tree. Classic profilers make another boundary primary, such as a function call stack. This paper argues that agent profiling needs a smaller abstraction: an `operation` is a weighted record with fields, and an `operation stack` is a recursive projection over those fields chosen at query time. Aggregation is not a separate object or a fixed trace tree; it is the result of choosing a stack shape over the same operations. This is not merely a claim about query-time aggregation, which classic profilers and trace processors also support in different forms. The scoped novelty is the combination of an agent-operation record model, recursive multi-field operation-stack projections, and a hidden-label cross-dataset localization benchmark.

We implement this model in the Rust AgentPPprof profiler and evaluate it on public labeled agent trajectories. Across 15 public sources, the profiler maps 47,590 samples into the same operation layer. On four oracle-rich families, AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human, we build six localization tasks over 34,539 operations and 3,699 positive labels. The main R320 experiment treats profile groups as ranked localization outputs and scores 144 view/ranker policies with hidden labels. Query-aware operation-stack profiling inspects a median 9.37% of operations in the top five groups, compared with 100% for flat summaries. Against fixed-session drilldown, operation stacks improve top-five recall on 5 of 6 tasks and reduce median group fragmentation from 285.0 to 157.5 groups. Query-aware ranking improves average precision over width-only operation-stack ranking on all six tasks. The Rust profile-spec implementation of operation-level rank features improves semantic-stack AP over width on 5 of 6 tasks. A reproducibility probe reruns 76 tracked profile specs and an opt-in deterministic-output mode makes all 76 raw profile artifacts byte-stable across repeated runs. A held-out transfer probe selects rank policies without using target-task labels; leave-task and leave-dataset selection both improve semantic/coarse AP over width on 4 of 6 tasks and semantic first-positive work on all six tasks. A paired-bootstrap uncertainty audit over the same six task families finds stable directions for the main AP, work, recall, and fragmentation comparisons while preserving flat-recall and fixed-session-work counterpoints. A label-permutation negative-control audit keeps visible rankings fixed and shows that operation-stack query-aware AP exceeds the prevalence/group-size null on all six tasks, while top-five precision and first-positive work remain limited. A view/depth task-fit audit shows that best visible AP is split across operation-stack, fixed-session, and dataset-native views, and that best top-five F1 spans five view families; no single hierarchy or source-task selector is enough. The result supports a profiling claim rather than a user-study claim: operation/operation-stack profiling can faithfully localize and rank task-relevant failures, quality problems, and semantic boundaries on real labeled traces, while exposing actionable tradeoffs among stack fields, mappings, rankers, and drilldown depth.

1 Introduction

AI-agent traces mix many objects that traditional profiling and tracing systems keep separate. A coding agent may ask a model for a plan, call a shell tool, launch a subagent, inspect a file, and retry a failing command. A web or desktop agent may click, type, observe a page, repeat an action, and receive a benchmark label saying that the action was unsafe, redundant, incorrect, or the start of a human subtask. If the profiler fixes the hierarchy at prompt, session, or span boundaries, these questions become hard to ask: which phases concentrate unsafe operations, which action families cause redundant steps, and which groups of operations cross sessions but share the same failure mode?

AgentPPprof takes the opposite position. The profiler stores a sequence as operations, where prompts, sessions, tool calls, processes, syscalls, GUI actions, and plan steps are all operation shapes or operation fields. Users then choose an operation stack to fold the same operations at different depths. One query may group by `task,phase,action`; another may group by `environment,safety`. Mapping and tagging are first-class field-derivation steps, but they do not add a third profiler object. They only write fields before the stack query runs. Thus the novelty is the abstraction that connects stack shape to aggregation: changing the stack changes what is comparable, localizable, and actionable without changing the underlying trace object model.

This paper makes a profiling claim, not a human-productivity claim. A top-tier profiling paper must show that the profiler faithfully attributes, localizes, and ranks problems on real workloads, and that the output leads to optimization insight with better accuracy or inspection cost than baselines. For agents, the claim is:

> *Operation/operation-stack profiling can accurately localize task-relevant failures, quality problems, and semantic boundaries in real labeled agent traces, while requiring less inspection work than flat summaries and less fragmentation than the fixed-session drilldown used here as a span-tree proxy.*

We evaluate this claim with public labeled traces rather than a human study. A human or agent analyst study would be useful for productivity claims, but it is not required to test profiler fidelity. Our evaluation therefore uses hidden labels as the oracle for ranked profile groups and asks seven research questions: RQ1 tests whether the two abstractions cover heterogeneous trajectories. RQ2 tests whether recursive stacks avoid fixed-boundary fragmentation. RQ3 tests whether mappings derive useful fields without creating extra objects. RQ4 tests whether folded boundaries align with human subtask labels. RQ5, the main R320 experiment, tests localization and ranking accuracy. RQ6 tests whether the profiler output produces actionable tuning decisions. RQ7 tests whether the profile-spec path is reproducible, byte-stable when requested, and has reportable local execution cost on the same tracked inputs.

2 Design

2.1 Operations

An `operation` is the only sample-level object in AgentPProf. It is a weighted record with string fields. The same representation can encode a prompt, a tool call, an API call, a browser action, a PyAutoGUI action, a process event, or a derived label such as `looping=yes` or `step_correct=incorrect`. External datasets enter the system as operation JSONL. Local Codex or Claude sessions can also be exported as an agent-session trace, imported back, or converted to operation JSONL before profiling.

2.2 Operation Stacks

An `operation stack` is an ordered list of fields. Folding operations by that list yields the usual folded-stack representation, but the stack is chosen at query time. The same operation file can be folded as a flat task summary, a fixed-session tree, a dataset-native hierarchy, a raw action stack, a semantic operation stack, or an oracle label-drilldown. Fixed sessions and spans are therefore baselines and exchange containers, not core profiler abstractions.

2.3 Mapping and Tagging

Mapping and tagging derive new operation fields before folding. For example, desktop actions such as `type` and `key` can map to `phase=input`, while tool/API rows can map to a tool phase before generic action rules run. The Rust CLI exposes inline rules, rule files, profile specs, operation predicates, and stack overrides. `–where` predicates run after mapping and before stack construction, implementing the query predicate without adding a profiler object. Profile specs record operation files, mappings, predicates, views, stacks, and outputs for reproducibility, while command-line flags can still override the stack query.

3 Evaluation Setup

Table 1 summarizes the public trace families. We do not create new test sets, sync complete image corpora, or depend on gated data. The 15 sources establish generality, while the main accuracy results use the four oracle-rich families with failure, safety, quality, and human-boundary labels.

For R320, we build six tasks: AgentRewardBench looping, AgentRewardBench side-effect, SATraj unsafe operation, AgentNet incorrect step, AgentNet redundant step, and OSWorld-Human group start. Each task has an oracle-positive field, but non-oracle rankers never read that field. We compare six views: flat, fixed-session, dataset-native hierarchy, raw action stack, semantic operation stack, and label-drilldown. We compare four rankers: width, visible-risk, query-aware, and oracle upper bound. Label-drilldown and oracle upper bound are marked as hidden upper bounds.

The metrics match a profiler paper's main claim. Fidelity uses precision@k, recall@operation-budget, F1@k, average precision as a block-level AUPRC-style score, nDCG over positive operation counts, and work-to-first-positive. Fragmentation uses group count, positive group count, and work or groups to reach 50% positive recall. Actionability uses per-task comparisons among stack fields, mapping choices, ranker choices, and oracle headroom. Reproducibility uses repeated Rust `agentpprof –profile-spec` executions over tracked specs and tracked operation JSONL inputs, checking semantic output hashes, sample counts, unique-stack counts, runtime, and output size. Build time is excluded from the profiler runtime measurement.

4 Results

4.1 RQ1: Do Two Abstractions Cover Heterogeneous Traces?

The 14 core public sources produce 42,590 operations. Adding the supplemental ScaleCUA stream brings the smoke set to 47,590 operations. These operations cover web, mobile, desktop, software, API, tool-agent-user dialogue, expert failure labels, safety labels, human grouped-action labels, and desktop step-quality labels. No source required a prompt-specific, GUI-specific, or safety-specific profiler object.

4.2 RQ2: Is Stack Depth a Query?

On the same 13,265 operations, a depth sweep folds to 9 dataset stacks, 57 phase stacks, 226 tool/semantic stacks, 455 action stacks, and 3,757 fixed-session stacks. A fixed session boundary therefore increases stack count by 417.4× relative to dataset depth. On AgentNet, a tracked profile spec folds 16,741 operations into 608 diagnostic stacks, while a CLI stack override changes the output to 83 stacks without changing the operation input. The result supports query-time recursive stack choice rather than a fixed prompt/session hierarchy. R321 adds an implementation probe for query predicates: profile specs over the tracked R300 operation file select 729/729, 714/714, and 4,285/4,285 expected operations with mapping-derived `where_rules` before folding. R322 adds the corresponding Rust JSON ranking surface: profile specs with visible `rank_rules` emit ranked operation-stack groups over the same six tasks, improving AP over width on 4 of 6 tasks while preserving counterexamples for richer group-level rankers. R323 adds a `rank_mode` probe: score-first ranking improves AP over width-boost on 4 of 6 tasks and reduces SATraj first-positive work from 0.6376 to 0.0842, while side-effect and OSWorld-Human remain counterexamples. R324 adds operation-level rank features: `rank_op_rules` match visible mapped `field=value` operation tokens and aggregate matched weight inside each folded group. On semantic stacks, AP improves over width on 5 of 6 tasks and first-positive work improves on 5 of 6 tasks; on coarser stacks, AP improves on 4 of 6 tasks while group counts fall. R325 replays the same scrubbed profiler input and stack projections with newly generated profile specs that remove one visible feature at a time. It finds 7 critical feature instances, 3 misleading feature instances, and a stack-depth tradeoff: coarse stacks are AP-preferred on 2 of 6 tasks but reduce group counts on all six tasks. R326 tests whether these rank policies are brittle hand-tuned weights. A global equal-weight visible feature bank improves AP over width on 4 of 6 semantic tasks and 5 of 6 coarse tasks. Equal-weight task policies stay within 0.02 AP of their weighted counterparts on 8 of 12 task/depth variants. R325-guided repairs improve AP on 2 of 3 misleading-feature cases and first-positive work on 2 of 3 cases; 1 of 3 improves both metrics. This is post-hoc actionability evidence, not a label-free deployment ranker. R329 adds a target-held-out transfer probe. Candidate policies include the global equal feature bank and six source-task equal-weight

Table 1: Public labeled trace families used by the evaluation. The main R320 accuracy benchmark uses the four oracle-rich rows in the bottom half.

Family	Native labels	Access path	Evaluation role
WebLINX, Mind2Web, AgentTrek, WebShop	Web actions, demos, tasks, rewards, action history	Hugging Face viewer or public shards	Web navigation coverage
API-Bank, ToolBench, tau-bench	API calls, tool messages, solution paths, outcomes	Hugging Face rows or repo JSONL	Tool/API and dialogue coverage
SWE-agent	Issue trajectory, command actions, target outcome	Hugging Face viewer	Software-agent coverage
AndroidControl, GUI-Odyssey, ScaleCUA	Mobile or GUI actions, instructions, history context	Public/HF samples	Mobile and GUI coverage
AgentRewardBench	Expert success, side-effect, looping, optimality	HF annotations plus cleaned BrowserGym steps	Failure and looping tasks
SATraj-OS	Desktop actions, safety, reward, attack type	HF safety config	Safety task
OSWorld-Human	Human single-action and grouped-action traces	Public GitHub JSON	Human-boundary task
AgentNet	Human desktop PyAutoGUI, step correctness, redundancy	HF repo JSONL streaming	Step-quality tasks

policies. Leave-task and leave-dataset selection use only non-target task scores, while target labels are reserved for final scoring. Both protocols improve semantic/coarse AP over width on 4 of 6 tasks and semantic first-positive work on 6 of 6 tasks, but side-effect, SATraj, and boundary counterexamples remain; the result isolates policy transfer from target-label tuning without claiming a universal ranker.

4.3 RQ3: Do Mappings Help Without Adding Objects?

Held-out mapping rules improve compression from 14.049 to 19.091 on 3,990 operations. Leave-dataset-out mapping reduces unique stacks on 6 of 9 held-out datasets with no negative stack-reduction regressions after API/tool phase precedence is fixed. A supervised OSWorld-Human boundary backend reaches F1 0.7735 on held-out adjacent pairs and writes predicted boundary fields that the Rust profiler folds as ordinary operation fields. Boundary backends therefore extend field derivation, not the profiler object model.

4.4 RQ4: Do Stacks Match Human Boundaries?

OSWorld-Human gives the strongest direct boundary oracle. On 320 exact-aligned tasks with 4,011 operations and 2,075 human groups, grouped-depth stacks reduce 482 action stacks to 106 grouped stacks. A conservative `group_pattern` projection has human-group boundary F1 0.627 with precision 1.0 and recall 0.456. This is not complete intent discovery, but it shows that the same single-action sequence can be folded at action depth or human-group depth and scored against a human boundary oracle.

4.5 RQ5: Does the Profiler Localize and Rank Labeled Problems?

Table 2 gives the main R320 accuracy result. Flat summaries recover all positives, but only by inspecting the whole task, so their median work@5 is 1.0 and their work-to-first-positive is 1.0. Fixed-session drilldown finds an early positive with median work-to-first-positive 0.0044 and median work@5 0.0163, but its median recall@5 is only 0.0233 and it fragments the workload into 285.0 groups. Query-aware operation stacks inspect median 0.0937 work in the top five groups, reach median recall@budget30 of 0.3900, and reduce the median group count to 157.5.

The strongest comparison is not universal dominance. Operation stacks improve top-five recall over fixed-session query-aware drilldown on 5 of 6 tasks and reduce group fragmentation from median 285.0 to 157.5 groups, but fixed-session often finds the first positive with less work. Operation stacks also save work against flat summaries on all tasks, but flat remains a complete full-recall counterpoint. Dataset-native hierarchy sometimes has higher recall, but at high inspection work. These mixed results are the desired profiling-paper evidence: operation stacks occupy a useful Pareto region between flat summaries and fixed-session drilldown.

R330 audits uncertainty over these R320 comparisons without rerunning the profiler or creating data. It bootstraps paired task-family deltas for 10,000 resamples. Against flat width, operation-stack query-aware has stable positive AP delta (mean 0.1219, 95% CI [0.0406, 0.2175]), stable lower top-five work (mean -0.8168, CI [-0.9653, -0.6568]), stable higher 30% budget recall (mean 0.4510, CI [0.3450, 0.6143]), and stable lower work-to-first-positive (mean -0.9303, CI [-0.9855, -0.8650]). Against fixed-session query-aware, operation stacks have stable higher top-five recall (mean 0.1801, CI [0.0542, 0.3127]), higher top-five F1 (mean 0.1702, CI [0.0549, 0.2870]), and fewer groups (mean -178.0, CI [-340.0, -39.0]). Against width-only operation stacks, query-aware ranking has stable higher AP (mean 0.0932, CI [0.0110, 0.2184]) and lower top-five work (mean -0.3101, CI [-0.4209, -0.1997]). The same audit marks flat top-five recall, fixed-session first-positive work, and raw-action mapping comparisons as mixed or counterpoints; it is a task-level uncertainty audit, not a per-operation independence or human-utility claim.

R331 adds a prevalence and group-size negative control. It keeps each visible policy's groups and ranking order fixed, then randomly reallocates each task's hidden positive operations across same-size group positions for 2,000 permutations. Operation-stack query-aware AP exceeds the 95% null on 6 of 6 tasks, with median observed-minus-null AP 0.0759; 30% budget recall exceeds

Table 2: R320 profiler accuracy benchmark. Hidden policies read oracle labels and are included only as upper bounds. WTFP is work-to-first-positive.

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	0	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	0	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	0	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	0	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	0	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	0	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	1	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	1	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

the null on 5 of 6 tasks, with median delta 0.0904. These results show that the main AP and budgeted-recall signals are not explained by prevalence and group sizes alone. The same audit is also a boundary: top-five precision exceeds the null on only 3 of 6 tasks, work-to-first-positive on 0 of 6, width-only operation-stack AP on 5 of 6, and fixed-session/raw-action AP on 6 of 6. R331 therefore supports mechanism isolation and baseline realism, not single-view dominance or a label-free deployment ranker.

R332 asks whether a single visible view/depth policy can be fixed after R320. It reuses the R320 visible policy scores, does not rerun profiling, and excludes oracle policies from selection. Best visible AP is split across operation-stack, fixed-session, and dataset-native views: 3 of 6, 2 of 6, and 1 of 6 tasks, respectively. Best top-five F1 is more dispersed: raw-action stacks win 2 of 6 tasks, while dataset-native, fixed-session, flat, and operation-stack each win 1 of 6. Operation-stack query-aware is best for AP on 3 of 6 tasks and best for 30% budget recall on 3 of 6 tasks, but only 1 of 6 for top-five F1 and 2 of 6 for work-to-first-positive. It still reduces group fragmentation relative to fixed-session query-aware on 4 of 6 tasks, with median group ratio 0.5543. Leave-task source selection does not solve view choice universally: selected-equals-best is 0 of 6 for AP and 0 of 6 for top-five F1. R332 therefore supports the design implication that view, stack fields, predicates, and rankers must remain query-time configuration over the same operations; fixed sessions, span trees, and dataset-native hierarchies remain baselines or drilldown views, not profiler abstractions.

4.6 RQ6: Are the Results Actionable?

R320 turns each task into an optimization diagnosis, summarized in Table 3. SATraj unsafe is the clearest operation-stack win: environment, phase, and action fields plus query-aware risk ranking give the best visible top-five F1. AgentRewardBench looping shows that `repeat_signal` is useful, but positives are prevalent, so the next optimization is prevalence-aware ranking. Side-effect has large oracle headroom, pointing to deeper write/input mappings. AgentNet step-quality tasks need action-family localization followed by fixed-session examples. OSWorld boundary tasks need boundary-derived fields rather than action depth alone. R322 moves a bounded version of the ranking policy into the Rust profiler surface. It shows that visible stack-text rules can be replayed from profile specs and scored after ranking, but it also exposes a limitation: SATraj safety and AgentRewardBench side-effect need group-level feature density rather than binary stack-text boosts alone. R323 turns that limitation into a rank-policy ablation. Switching the same visible rules from width-boosted ranking to score-first ranking improves AP on 4 of 6 tasks and top-five lift on 4 of 6 tasks, but hurts side-effect and OSWorld-Human, which points to deeper side-effect mappings and boundary-derived fields rather than one universal ranker. R324 closes more of that implementation gap by moving group-feature ranking into the Rust profiler. The profile spec records `rank_op_rules`; the R324 harness first derives a scrubbed visible-operation profiler input from the R300 source, so the profiler matches visible operation fields before folding, and aggregates feature density per stack group. Hidden labels are not passed to Rust and are used only for offline scoring. Semantic-stack operation-feature ranking improves AP over width on 5 of 6 tasks, top-five lift on 4 of 6 tasks, and first-positive work on 5 of 6 tasks. The coarse-stack variant improves AP on 4 of 6 tasks while reducing group fragmentation, but OSWorld-Human remains a counterexample that needs boundary-derived fields. R325 then asks which knobs matter. Leave-one-feature ablation identifies `repeat_signal` as critical for looping, write-action features as critical for side effects, and `status=success` as the strongest SATraj safety feature. It also identifies misleading features: a SATraj loop-like rule and OSWorld-Human input-phase rules can hurt AP or first-positive work. This is the concrete actionability result: the profiler points to which field mappings, rank features, and stack depths should be changed for each task family. R326 checks that this actionability is not only a single weight vector. Global equal visible features beat width on 4 of 6 semantic and 5 of 6 coarse tasks, task-equal policies remain within 0.02 AP of weighted policies on 8 of 12 variants, and R325-guided repairs improve AP on 2 of 3 misleading-feature cases and first-positive work on 2 of 3 cases, with 1 of 3 improving both. The repair result is intentionally scoped: it uses offline ablation feedback to show how profiler output can guide policy changes, not to claim an automatically learned universal ranker. R329 checks a stronger leakage boundary for policy selection. The profiler evaluates 96 policy/task/stack combinations, then selects a transfer policy using leave-target-task or leave-dataset scores before looking at the target labels. Held-out scoring improves semantic/coarse AP over width on 4 of 6 tasks and semantic first-positive work on all six tasks. The mean AP gap to the oracle-best candidate is about 0.032, which leaves headroom for task-family selection and calibration. The supported conclusion is mechanism isolation: operation-level rank features and source-task policies transfer enough to guide tuning, while still requiring task-aware choice rather than a fixed universal ranker. R332 makes the same point at the view/depth level: best visible AP and best top-five F1 are not monopolized by one hierarchy, and leave-task source selection cannot reliably pick the best visible view. Actionability is therefore not a hard-coded

"best tree"; it is the ability to change stack depth, field mapping, predicates, and rank policy while scoring the same operations.

4.7 RQ7: Are Profile Specs Reproducible with Reportable Cost?

R327 reruns the existing profile-spec artifacts used by R300, R324, and R326. The workload contains 76 tracked specs: four view specs, 12 operation-feature rank specs, and 60 robustness specs. Each spec is executed twice by the Rust profiler against tracked operation JSONL inputs, for 152 profiler invocations. No dataset is downloaded or synced. The determinism gate hashes the semantic profile content after removing the JSON `generated_at` timestamp, and separately reports raw-byte hashes.

All 76 specs reproduce the same semantic profile hash, sample count, and unique-stack count across repetitions. Raw-byte determinism is only 4 of 76, because JSON outputs intentionally include a generation timestamp; the stable semantic hash keeps the reproducibility claim on the profile content. Median output size is 37,546 bytes, the largest output is 306,099 bytes, and the largest profile has 2,012 unique stacks. This supports an offline artifact and profile-spec reproducibility claim, not live eBPF overhead or human utility.

R328 closes the raw-artifact caveat with an explicit deterministic-output mode. The Rust CLI accepts `–deterministic-output`, and profile specs may set `deterministic_output`; the mode replaces JSON `generated_at` and pprof profile time with fixed values while leaving profile content unchanged. R328 reruns the same 76 specs and 152 invocations with this flag. Both semantic and raw-byte determinism are 76 of 76. The median runtime in this run is 1,577.5383 ms and the p95 is 2,730.9206 ms; we report these as artifact costs for this run, not as a performance comparison with R327.

5 Related Work and Novelty

Classic profiling and trace analysis. Flame graphs and pprof establish the folded-stack lineage for profiling: profiles consist of weighted samples attached to a location hierarchy, can carry sample labels, and pprof can visualize labels as pseudo stack frames through tag-root or tag-leaf options [1, 2]. Perfetto generalizes trace ingestion and SQL analysis over many trace formats and can derive new events from trace contents [3]. These systems are closer than a simple "fixed hierarchy" baseline. AgentPProf therefore does not claim novelty for query-time aggregation alone. The difference is the domain object and evaluation target: the sample is an agent operation, the frames are recursive multi-field operation projections, and the groups are scored against hidden agent-trajectory labels across datasets.

Agent tracing and LLM observability. OpenTelemetry GenAI and OpenInference standardize GenAI spans, LLM calls, agent reasoning steps, tool invocations, retrieval operations, MCP, and provider-specific telemetry [4, 5]. LangSmith, Langfuse, and Phoenix expose traces, sessions, observations, evaluations, dashboards, datasets, and prompt iteration workflows [6–8]. AgentOps gives the closest academic same-problem framing by arguing that agent observability should trace goals, plans, tools, artifacts, and associated data [9]. These systems are strong prior work, so the safe novelty claim is not generic "agent observability." In this paper, traces, spans, sessions, and observations are exchange containers or baseline shapes. R320 evaluates fixed-session drilldown as the current span-tree proxy; real OpenTelemetry/OpenInference or Phoenix span-tree imports remain future interoperability baselines. The profiler abstractions are only operations and operation stacks, and the central mechanism is that stack shape determines semantic aggregation at query time.

Public labeled agent trajectories. AgentRewardBench, OSWorld-Human, OpenCUA/AgentNet, SATraj-OS, and OSWorld provide the labels and workloads that make R320 possible [12, 18–21]. Prior benchmark papers use these labels to evaluate agents, reward models, safety behavior, or efficiency. We use them as hidden oracles for a profiler benchmark: different stack-shaped aggregates become ranked outputs, and the evaluation measures precision, recall, AP, nDCG, inspection work, fragmentation, and optimization insight. This is the paper's scoped novelty.

6 Discussion

The result scope is deliberately narrow. R320 supports profiler fidelity, localization/ranking accuracy, work tradeoffs, fragmentation reduction, and actionable optimization insight on six real labeled tasks. R327 supports offline profile-spec reproducibility and local execution cost for the tracked artifacts; R328 adds opt-in byte-stable output for those artifacts. R329 adds held-out rank-policy transfer evidence, but selection still uses labeled training tasks. R330 adds task-paired uncertainty evidence over R320 point estimates, but it is not a per-operation independence analysis. R331 adds a label-permutation negative control for prevalence and group-size explanations, but it does not upgrade top-five precision, first-positive work, or label-free deployment claims. R332 adds a view/depth task-fit audit and shows that best visible policies vary by task and objective, so it supports a configurable analysis surface rather than an automatic view selector. These results do not show that humans or agents answer questions faster with the tool, nor do they measure live eBPF overhead. They also do not show automatic discovery of all intent boundaries, nor complete compatibility with OpenTelemetry, Phoenix, LangSmith, Langfuse, or Perfetto producers. Those systems remain related work and baselines. In this paper, trace formats are containers; fixed sessions are the evaluated drilldown baseline, while real span-tree imports remain future baselines. The profiler abstractions remain only operations and operation stacks.

The negative results are also useful. Mapping beats a raw action/status stack on top-five F1 for only 2 of 6 tasks, so field derivation is task-sensitive. Query-aware ranking improves AP over width-only operation-stack ranking on all six tasks, but top-five F1 and work still expose calibration and prevalence counterexamples. R322's Rust `rank_rules` improve AP over width on 4 of 6 tasks, R323's `rule-score` mode improves AP over `width-boost` on 4 of 6 tasks, R324's operation-feature ranking improves semantic-stack AP over width on 5 of 6 tasks, R325 finds 7 critical and 3 misleading feature instances, and R326 shows equal-weight/global-bank policies often preserve the gains while ablation-guided repairs improve AP on 2 of 3 misleading cases and first-positive work on 2 of 3 cases. R329 held-out transfer improves AP on 4 of 6 semantic and 4 of 6 coarse tasks and semantic first-positive work on 6 of 6 tasks, but still leaves

Table 3: R320 actionability summary. Each row identifies the stack or ranker change suggested by the profiler accuracy results.

Task	Best visible policy	Optimization insight
Looping	dataset-native:query-aware	Keep repeat-signal fields but add prevalence-aware ranking because positives are common.
Side-effect	fixed-session:width	Increase write/input weights or use a deeper side-effect mapping before ranking.
Unsafe operation	operation-stack:query-aware	Use environment, phase, and action stack fields and prioritize risky write actions.
Incorrect step	raw-action:width	Use desktop action family first, then drill into fixed sessions for examples.
Redundant step	raw-action:width	Raw action/status currently beats mapped stack depth, so tune quality-specific fields.
Group start	flat:width	Add boundary-derived fields because action-depth stacks fragment starts.

Table 4: R327 profile-spec reproducibility and offline execution cost. Runtime is seconds per spec, measured after the Rust binary exists. Semantic hashes exclude the JSON `generated_at` field; raw-byte hashes are reported separately.

Workload	Specs	Med. s	P95 s	Med. stacks
R300 view specs	4	3.448	4.273	631.0
R324 rank-feature specs	12	1.539	2.692	41.0
R326 robustness specs	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

Table 5: R327/R328 determinism checks over the same 76 profile specs. R328 uses `--deterministic-output`.

Run	Mode	Sem.	Raw	Med. s	P95 s
R327	default	76/76	4/76	1.581	2.719
R328	deterministic	76/76	76/76	1.578	2.731

side-effect, SATraj, and boundary-field counterexamples. R330 classifies 10 task-paired metric checks as directionally stable and 10 as mixed or counterpoints, which keeps the actionability claim auditable rather than absolute. R331 shows operation-stack query-aware AP is above the label-permutation null on 6 of 6 tasks and budgeted recall on 5 of 6, but top-five precision and first-positive work remain null-limited. R332 shows that best AP and best top-five F1 are split across multiple view families, and that leave-task source selection is exact-best on neither AP nor top-five F1. The right design implication is to expose stack fields, mapping rules, rank modes, operation-level rank features, global defaults, and task-specific ranker policies to the user, not to hard-code one hierarchy.

7 Conclusion

AgentPProf reduces agent profiling to two abstractions. Operations carry all observed and derived fields, while operation stacks fold those fields at the depth chosen by the query. Mapping and `--where` predicates choose fields and operation subsets before folding, but they remain query components rather than profiler objects. Across 15 public labeled trace sources, this model covers web, mobile, desktop, software, API, dialogue, failure, safety, human boundary, and step-quality trajectories. On the four oracle-rich families used for R320, operation-stack profiling localizes and ranks real labeled problems with less work than flat summaries and less fragmentation than fixed-session drilldown. R324–R329 show that visible operation-level rank features, feature ablations, equal-weight/global-bank policies, ablation-guided repairs, and target-held-out transfer can turn those rankings into concrete optimization guidance while preserving hidden-label separation. R330–R332 then stress-test the same comparisons with task-paired uncertainty, label-permutation negative controls, and view/depth task-fit audits. The paper's central claim is therefore a profiler claim: the operation/operation-stack model faithfully exposes task-relevant failures, quality problems, and semantic boundaries, and it produces actionable optimization insight without relying on prompt/session/span boundaries. R327 adds that these claims are carried by repeatable profile specs whose semantic outputs reproduce across runs on tracked inputs; R328 makes the same tracked profile outputs byte-stable when deterministic artifact mode is enabled.

References

[1] Brendan Gregg. Flame Graphs. https://www.brendangregg.com/flamegraphs.html.
[2] Google pprof documentation. https://github.com/google/pprof/blob/main/doc/README.md.
[3] Perfetto Trace Processor documentation. https://perfetto.dev/docs/analysis/trace-processor.
[4] OpenTelemetry GenAI semantic conventions. https://github.com/open-telemetry/semantic-conventions-genai.
[5] OpenInference specification. https://arize-ai.github.io/openinference/spec/.
[6] LangSmith Observability. https://docs.langchain.com/langsmith/observability.
[7] Langfuse Observability. https://langfuse.com/docs/observability/overview.
[8] Arize Phoenix documentation. https://arize.com/docs/phoenix.
[9] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling Observability of LLM Agents. https://arxiv.org/abs/2411.05285.
[10] McGill-NLP WebLINX. https://huggingface.co/datasets/McGill-NLP/WebLINX.
[11] OpenGVLab GUI-Odyssey. https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey.
[12] WukLab OSWorld-Human. https://github.com/WukLab/osworld-human.
[13] xlangai AgentNet. https://huggingface.co/datasets/xlangai/AgentNet.
[14] McGill-NLP AgentRewardBench. https://huggingface.co/datasets/McGill-NLP/agent-reward-bench.
[15] AI45Research SATraj-OS. https://huggingface.co/datasets/AI45Research/SATraj-OS.
[16] AgentSuite tau-bench trajectories. https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories.
[17] OpenGVLab ScaleCUA-Data. https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data.
[18] Xing Han Lù et al. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. https://arxiv.org/abs/2504.08942.
[19] Tianbao Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. https://arxiv.org/abs/2404.07972.
[20] Xinyuan Wang et al. OpenCUA: Open Foundations for Computer-Use Agents. https://arxiv.org/abs/2508.09123.

[21] Shanghai AI Lab. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. https://arxiv.org/abs/2605.06230.